

Face Verification & Identification - LFW Dataset

CIE 555 — Neural Networks and Deep Learning Spring 2026
Assignment 2 — Technical Report

Mohammed Ali | ID: 202200594

1. Introduction

This report documents the design, training, and comparative evaluation of Siamese Network architectures for face verification on the **Labeled Faces in the Wild (LFW)** dataset. Two backbone strategies are contrasted — a Custom CNN trained from scratch and a transfer-learning backbone based on **EfficientNetB0** — each trained under two metric-learning loss regimes: **Contrastive Loss** (pair-based) and **Triplet Loss** (triplet-based). All loss functions are implemented manually from scratch. Evaluation covers training and validation loss/accuracy curves, Euclidean distance distribution histograms, ROC curves, and an end-to-end inference pipeline.

2. Data Engineering & Filtering Strategy

The raw LFW directory was parsed programmatically by iterating over every identity folder and recording its image count. A two-sided count filter was then applied:

- **Lower bound** (`MIN_IMAGES = 5`): Persons with fewer than 5 images are excluded. With only 3 images a person can form at most 3 positive pairs — too sparse to learn appearance variance within a single identity. Raising the threshold from 3 to 5 roughly triples the positive-pair yield per person.
- **Upper bound** (`MAX_IMAGES = 50`): Persons with more than 50 images are capped. Without this, celebrities (e.g., George W. Bush with 530 images) would dominate pair generation and bias the network toward a handful of identities.

Eligible identities were split at the **person level** (open-set protocol) using a 70 / 20 / 10 ratio (train / val / test), ensuring no test identity is ever seen during training. Negative diversity was enforced by cycling through a shuffled pool of non-self identities when assigning negatives to each anchor.

Table 1: Open-set person-level dataset splits.

Split	Identities	Pairs / Triplets per Person	Pos : Neg
Train	~756	40 + 40	1 : 1
Validation	~216	40 + 40	1 : 1
Test	~108	40 + 40	1 : 1

3. Augmentation: Online vs. Offline

This project uses **online (on-the-fly) augmentation** via the `tf.data` pipeline, applied only during training.

Transforms applied: (i) random crop + resize (0.85–1.0 fraction) to simulate head-pose and zoom variation; (ii) random horizontal flip — faces are bilaterally symmetric; (iii) brightness ± 0.15 and

Table 2: Offline vs. online augmentation comparison.

Dimension	Offline	Online (used here)
Execution	Pre-generate & save to disk	Generated at runtime each epoch
Diversity	Fixed copies, same every epoch	New random transforms each epoch
Disk usage	High — multiplies dataset size	Near-zero — no extra files
Regularisation	Moderate; copies can be memorised	Strong; true stochastic regulariser

contrast [0.85, 1.15] to model lighting changes; (iv) Gaussian noise ($\sigma = 0.008$) as a lightweight regulariser. Colour-jitter is deliberately omitted — skin-tone is a valid identity cue.

4. Model Architectures

4.1 Custom CNN Backbone

The custom backbone is a four-block convolutional network built entirely from scratch. Each block uses two consecutive 3×3 convolutions (Pattern B), followed by BatchNormalization, MaxPooling(2×2), and Dropout. Filter depths double per block: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$.

- **Activation — LeakyReLU ($\alpha = 0.1$):** Prevents dying neurons in the deep embedding space; standard ReLU caused silent unit collapse in early experiments.
- **Pooling — Global Average Pooling (GAP):** Reduces parameters from ~ 16 M (Flatten) to ~ 800 K, acting as a structural regulariser critical for small datasets.
- **Embedding head:** Two dense layers project to 128-D followed by an L2NormalizeLayer, bounding all pairwise Euclidean distances to $[0, 2]$.
- **Dropout:** $0.10 \rightarrow 0.15 \rightarrow 0.15 \rightarrow 0.20$ across blocks with 0.20 on the embedding dense layer.

Loss & accuracy — Contrastive: The training loss descends steeply over the first 8–10 epochs then plateaus. Validation loss closely tracks training loss, confirming good generalisation. **Contrastive accuracy** is computed by thresholding the output Euclidean distance at $d = 0.7$: pairs with $d < 0.7$ are labelled the same person. This gives a simple, human-readable classification accuracy. Test result: **78.77 %** (test loss 0.1491; best val loss 0.1379 at epoch 36).

Loss & accuracy — Triplet: Triplet loss starts near 0.48 and decreases slowly through an easy-triplet plateau before semi-hard examples drive further learning. **Triplet accuracy** is the fraction of test triplets satisfying $d(a, p) < d(a, n)$ — a threshold-free metric that directly measures embedding geometry correctness. Test result: **87.05 %** (test loss 0.1744; best val loss 0.1881 at epoch 38).

4.2 Transfer Learning Backbone — EfficientNetB0

EfficientNetB0 was chosen over MobileNetV2 for its compound scaling strategy (jointly optimising depth, width, and resolution), yielding stronger ImageNet features (~ 5.3 M parameters). Training follows a two-phase protocol:

- **Phase 1 — Head warm-up (20 epochs, $\text{LR} = 3 \times 10^{-4}$):** The entire base is frozen; only the 256-D dense head and 128-D embedding layer are trained. This prevents catastrophic forgetting before any fine-tuning begins.
- **Phase 2 — Selective fine-tuning (25 epochs, $\text{LR} = 1 \times 10^{-5}$):** The top 50 layers are unfrozen. The very low learning rate allows gentle domain adaptation from ImageNet to LFW without destroying general-purpose features.

Loss & accuracy — Contrastive: The combined loss curve shows a two-segment shape: a moderate descent in Phase 1 followed by a steady drop in Phase 2 as fine-tuned features refine the embedding space. During Phase 1, validation loss falls *below* training loss — typical for a frozen base with an easily generalisable head. **Contrastive accuracy** reaches **80.43 %** on the test set (test loss 0.1355; best val loss 0.1434).

Loss & accuracy — Triplet: The best-performing combination overall. Phase 2 fine-tuning is especially effective here: the relative-distance objective steers pretrained features toward identity-discriminative dimensions more aggressively than the absolute threshold of contrastive loss. The loss curve shows the smallest train/val gap across all four models. **Triplet accuracy** reaches **88.97 %** — the best result across all experiments (test loss 0.1588; best val loss 0.1616) — with the tightest intra-class clustering in the distance distribution plots.

Table 3: Architecture comparison.

Property	Custom CNN	EfficientNetB0
Depth	4 Conv blocks (32 $\rightarrow$ 256)	EfficientNetB0 + 2 dense layers
Parameters	$\sim$ 800 K trainable	$\sim$ 5.3 M base + $\sim$ 200 K head
Activation	LeakyReLU ($\alpha = 0.1$)	Swish (default)
Regularisation	BatchNorm + Dropout 0.10–0.20	Dropout 0.35 on head
Embedding	128-D, L2-normalised	128-D, L2-normalised
Training	Single phase, LR = 10^{-3}	Two-phase: $3 \times 10^{-4} \rightarrow 10^{-5}$

5. Loss Function Implementation

Both loss functions are implemented as custom TensorFlow callables — no built-in Keras losses were used.

5.1 Contrastive Loss (margin $M = 1.0$)

$$\mathcal{L} = y \cdot d^2 + (1 - y) \cdot \max(M - d, 0)^2 \quad d = \text{Euclidean distance}, \quad y \in \{0, 1\}$$

When $y = 1$ (same person) the loss minimises the distance directly. When $y = 0$ (different persons) it penalises distances smaller than M . **Contrastive accuracy** applies a fixed threshold $d = 0.7$, giving a straightforward classification accuracy over all test pairs.

5.2 Triplet Loss (margin $\alpha = 0.5$)

$$\mathcal{L} = \max(d(a, p) - d(a, n) + \alpha, 0)$$

True Euclidean distance (not squared) is used — L2-normalised embeddings live on a unit hypersphere where squared distances distort gradients near the poles. **Triplet accuracy** is the fraction of triplets satisfying $d(a, p) < d(a, n)$, a threshold-free metric that directly reflects embedding geometry without requiring a fixed decision boundary.

Table 4: Contrastive vs. Triplet loss comparison.

Criterion	Contrastive	Triplet
Data unit	Positive / negative pair	Anchor–Positive–Negative triplet
Gradient signal	Absolute distance to threshold	Relative distance difference
Accuracy metric	Threshold at $d = 0.7$	Fraction $d(a, p) < d(a, n)$
Collapse risk	Low — direct push/pull	Higher; needs diverse negatives
Key advantage	Stable, interpretable boundary	Stronger metric geometry learning

6. Distance Distribution Analysis

For every model, Euclidean distances are computed over all test pairs and plotted as overlapping histograms — same-person pairs vs. different-person pairs — with a dynamic threshold at the midpoint of the two means.

- **Separation gap:** A wide gap between peaks indicates a clear identity boundary. Overlap signals confusion between visually similar but distinct identities.
- **Positive-pair spread:** A narrow same-person distribution means consistent embeddings across pose and lighting — the most important real-world property.
- **Custom CNN:** Moderate overlap. The contrastive variant shows a wider positive-pair tail (distances up to 0.9); the triplet variant gives tighter clustering.
- **EfficientNetB0:** Well-separated distributions in both settings. The triplet model produces the tightest intra-class cluster and the largest inter-class gap across all four experiments.

7. Results & Discussion

Table 5: Test-set results across all model-loss combinations.

Model	Loss	Test Acc.	Test Loss	Best Val Loss	Dist. Sep.
Custom CNN	Contrastive	78.77 %	0.1491	0.1379	Moderate
Custom CNN	Triplet	87.05 %	0.1744	0.1881	Good
EfficientNetB0	Contrastive	80.43 %	0.1355	0.1434	Good
EfficientNetB0	Triplet	88.97 %	0.1588	0.1616	Excellent

Contrastive vs. Triplet: Contrastive loss converges faster and is easier to tune. Triplet loss takes longer to escape easy-triplet plateaus but produces superior metric geometry, delivering an 8–10 point accuracy gain across both backbones. Notably the triplet loss is higher in absolute value than contrastive loss but this is expected — it measures a margin-based distance difference, not a bounded classification error.

Custom CNN vs. EfficientNetB0: Under contrastive loss, the EfficientNetB0 backbone improves accuracy by only 1.66 points (78.77 % $\rightarrow$ 80.43 %), much less than anticipated. However under triplet loss the advantage grows to nearly 2 points (87.05 % $\rightarrow$ 88.97 %). This suggests pretrained features matter most when combined with a loss that better exploits the geometry of the embedding space. The two-phase fine-tuning protocol is essential — skipping Phase 1 caused validation loss to spike, indicating catastrophic forgetting of the projection head.

Best overall — EfficientNetB0 + Triplet (88.97 %): This combination achieves the lowest test loss (0.1588) and highest accuracy. Its distance distribution shows the tightest intra-class clustering and widest inter-class margin across all four experiments, making it the recommended configuration for deployment. AUC values are available from the ROC curve visualisation in the notebook (Cell 38).